

Como Python pode combinar análise de dados e automação para transformar uma tarefa manual e repetitiva em um processo automatizado?

Thoughts

A combinação de análise de dados com automação em Python permite transformar uma tarefa manual, demorada e sujeita a erros operacionais em um fluxo de trabalho autônomo, padronizado e executado periodicamente sem intervenção humana.

1. Como o Processo Manual é Transformado

Em um cenário manual típico, um profissional precisa diariamente abrir planilhas no Excel, conferir se há células vazias ou erros de digitação, aplicar filtros condicionais, copiar subconjuntos de dados, realizar somas agrupadas em tabelas dinâmicas e salvar um novo arquivo para enviar à gestão.

O Python substitui essa rotina encadeando todas essas ações em um script executável (.py), estruturado em cinco etapas lógicas:

Leitura e Ingestão de Dados: O Python abre a planilha eletrônica original através da biblioteca Pandas e do motor OpenPyXL.

Higienização de Qualidade: O script valida tipos de dados, converte entradas de texto incorretas em colunas numéricas com `pd.to_numeric(..., errors="coerce")` e trata dados ausentes ou duplicados.

Análise e Regras de Negócio: Os dados são filtrados por critérios de status e consolidados com o método `.groupby()`, que aplica somas e estatísticas acumuladas.

Exportação do Relatório Final: O resultado processado é gravado diretamente em um novo arquivo Excel (.xlsx) via `.to_excel()`, mantendo a base original intocada.

Agendamento Autônomo: O script .py é cadastrado no Agendador de Tarefas do Windows (ou cron no Linux/Mac) para ser disparado automaticamente em horários programados ou ao ligar o computador.

2. Exemplo Prático Integrado

Abaixo está a demonstração do script de automação (rotina_financeira.py) que executa o tratamento e a consolidação de uma planilha de pagamentos:

```
import pandas as pd

# 1. Leitura: Carrega o arquivo Excel original para a memória
dados = pd.read_excel("pagamentos.xlsx")

# 2. Qualidade: Converte textos inválidos na coluna 'Valor' em nulos (NaN)
dados["Valor"] = pd.to_numeric(dados["Valor"], errors="coerce")

# 3. Filtragem: Isola apenas os registros que possuem Status 'Pendente'
pendentes = dados[dados["Status"] == "Pendente"]

# 4. Análise/Agrupamento: Agrupa por 'Unidade' e calcula a soma total
resumo_unidades = (
    pendentes
    .groupby("Unidade", as_index=False)["Valor"]
    .sum()
    .rename(columns={"Valor": "Valor_Total_Pendente"})
)

# 5. Exportação: Gera a nova planilha com os resultados consolidados
resumo_unidades.to_excel("relatorio_pendencias_unidades.xlsx", index=False)
```

Explicação passo a passo do código:

`pd.read_excel("pagamentos.xlsx")`: Ingesta os dados brutos utilizando o suporte de leitura do OpenPyXL.

`pd.to_numeric(..., errors="coerce")`: Garante a integridade da análise transformando falhas de digitação em NaN para evitar que o script pare com erros de execução.

`pendentes = dados[dados["Status"] == "Pendente"]`: Aplica a seleção condicional dos dados na memória.

`.groupby("Unidade", as_index=False)["Valor"].sum()`: Realiza a abordagem split-apply-combine, somando os valores financeiros por cada filial.

`.to_excel(..., index=False)`: Grava a nova planilha `.xlsx` sem incluir a coluna de índices numéricos do Pandas.

3. Torne a Execução Totalmente Automática (Sem Abrir o Editor)

Para que a rotina rode sozinha diariamente sem que você precise abrir o editor de código:

Garanta a extensão `.py`: Caso o desenvolvimento seja feito no Jupyter Notebook, exporte o arquivo via menu File -> Download as -> Python (`.py`).

Configure o Agendador de Tarefas do Windows:

Abra o Agendador de Tarefas e clique em Criar Tarefa Básica.

Defina a frequência (ex.: Diariamente às 08:00 ou Ao iniciar o computador).

Selecione a ação Iniciar um Programa e aponte o caminho do executável do interpretador Python (do seu ambiente virtual).

No campo Adicionar argumentos, insira o nome do seu arquivo `.py` e informe a pasta de início onde o script está salvo.

Com essa configuração, o sistema operacional invocará o Python no horário programado, executando a leitura, a limpeza, a análise e a geração do relatório em segundo plano sem que nenhuma janela precise ser aberta manualmente.

4. Vantagens da Abordagem

Eliminação de Erros Manuais: Regras de negócio e rotinas de qualidade passam a ser executadas com 100% de consistência.

Reprodutibilidade e Auditabilidade: O código-fonte documenta exatamente todas as transformações aplicadas nos dados.

Economia de Tempo: Tarefas que consumiriam horas de trabalho diário passam a ser concluídas em poucos segundos.